



南開大學
Nankai University

计算机学院
数据安全课程实验报告

零知识证明实践

姓名：孔祥昊

2311439

专业：计算机科学与技术

2026 年 4 月 23 日

目录

1 实验要求	2
2 实验原理	2
2.1 zk-SNARK 的核心特性	2
2.2 zk-SNARK 的工作原理	2
2.2.1 将计算问题转化为算术电路	2
2.2.2 构建一阶约束系统 (R1CS)	2
2.2.3 转化为二次算术程序 (QAP)	3
2.2.4 可信设置 (Trusted Setup)	3
2.2.5 证明生成与验证	3
3 实验	4
3.1 common.hpp: 算术电路与 R1CS 约束定义	4
3.2 mysetup.cpp: 可信设置	5
3.3 myprove.cpp: 证明生成	6
3.4 myverify.cpp: 证明验证	7
4 结果及验证	8
5 Github 链接	9

1 实验要求

- 参考教材实验 3.1, 假设 Alice 希望证明自己知道如下方程 $x^3 + x + 5 = out$ 的解。其中, out 是大家都知道的一个数。这里假设 out 的值为 35, 则 $x = 5$ 就是方程的解。请实现代码完成证明生成和证明的验证。

2 实验原理

零知识简洁非交互式知识论证 (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge, 简称 **zk-SNARK**) 是当前应用最广泛的零知识证明技术之一。它通过一系列复杂的数学变换, 将一个计算问题的正确性证明, 转化为一个极其简短且可快速验证的密码学证明。

2.1 zk-SNARK 的核心特性

- 零知识 (Zero-Knowledge):** 证明过程不会泄露任何关于秘密输入 (witness) 的信息。
- 简洁 (Succinct):** 生成的证明非常小 (通常仅几百字节), 且验证时间极短 (毫秒级), 与原始计算的复杂度无关。
- 非交互式 (Non-Interactive):** 证明者只需生成一个证明并发送给验证者, 无需像早期协议那样进行多轮交互。
- 知识论证 (Argument of Knowledge):** 证明者必须 “知道” 一个有效的秘密输入才能生成证明, 而不仅仅是让验证者相信陈述为真。

2.2 zk-SNARK 的工作原理

zk-SNARK 的工作流程可以分解为几个关键步骤, 其核心思想是将一个复杂的计算问题转化为一个关于多项式的数学问题。

2.2.1 将计算问题转化为算术电路

任何计算都可以被分解为一系列基本的加法和乘法运算。zk-SNARK 首先将需要证明的计算 (例如, “我知道一个数字 x , 使得 $x^2 + 3 = 7$ ”) 表示为一个**算术电路**。这个电路由代表加法和乘法的 “门” 组成, 并通过 “线” 连接。

2.2.2 构建一阶约束系统 (R1CS)

算术电路随后被转化为一阶约束系统 (Rank-1 Constraint System, R1CS)。R1CS 将电路中的每个门都表示为一个向量约束。其标准形式为:

$$\langle \mathbf{a}, \mathbf{s} \rangle \cdot \langle \mathbf{b}, \mathbf{s} \rangle = \langle \mathbf{c}, \mathbf{s} \rangle \quad (1)$$

其中:

- $\mathbf{s}$ 是一个包含所有输入、输出和中间变量的向量, 称为**见证 (Witness)**。
- $\mathbf{a}, \mathbf{b}, \mathbf{c}$ 是定义约束的系数向量。

- $\langle \cdot, \cdot \rangle$ 表示向量的内积。

例如，对于一个乘法门 $x \cdot y = z$ ，我们可以构造向量使得 $\mathbf{s}$ 中包含 x, y, z ，而 $\mathbf{a}, \mathbf{b}, \mathbf{c}$ 则分别选取 x, y, z 的系数，从而满足上述等式。

2.2.3 转化为二次算术程序 (QAP)

这是将离散的约束系统转化为连续多项式问题的关键一步。通过拉格朗日插值法，R1CS 中的向量 $\mathbf{a}, \mathbf{b}, \mathbf{c}$ 被转化为一组多项式 $A(x), B(x), C(x)$ 。这样，所有的约束就可以被一个核心的多项式方程来表示：

$$A(x) \cdot B(x) - C(x) = H(x) \cdot Z(x) \quad (2)$$

其中：

- $A(x), B(x), C(x)$ 是由见证 $\mathbf{s}$ 和 R1CS 约束共同决定的多项式。
- $Z(x)$ 是一个公开的目标多项式，其根对应于电路中的每一个约束。
- $H(x)$ 是一个商多项式。

这个方程的意义在于：如果证明者知道一个正确的见证 $\mathbf{s}$ ，那么他就能构造出 $A(x), B(x), C(x)$ ，使得 $A(x) \cdot B(x) - C(x)$ 能够被 $Z(x)$ 整除。证明的核心就变成了证明这个多项式整除关系成立。

2.2.4 可信设置 (Trusted Setup)

这是一个至关重要且存在潜在风险的初始化阶段。为了在不泄露秘密信息的前提下验证多项式关系，需要生成一组公共参数，即**公共参考字符串 (Common Reference String, CRS)**。

这个过程会生成一对密钥：

- **证明密钥 (Proving Key, pk)**：用于生成证明。
- **验证密钥 (Verifying Key, vk)**：用于验证证明。

在生成过程中会产生一些临时的随机数，被称为“**有毒废料 (Toxic Waste)**”。这些废料必须被安全地销毁。如果任何人保留了这些废料，他就可以伪造出看似有效的虚假证明。为了降低风险，通常会举行多方计算仪式 (MPC Ceremony)，让多个参与者共同生成 CRS，只要其中至少有一人是诚实的并销毁了自己的那部分随机数，整个系统就是安全的。

2.2.5 证明生成与验证

在可信设置完成后，证明者和验证者就可以开始工作：

1. **证明生成 (Prover)**：证明者拥有秘密见证 w 和公开输入 x 。他使用证明密钥 pk 和这些输入，通过椭圆曲线密码学等工具，计算出一个简短的证明 π 。这个证明本质上是对多项式 $A(x), B(x), C(x)$ 在某个秘密点上的“承诺”，并证明了它们满足 QAP 方程。
2. **验证 (Verifier)**：验证者收到公开输入 x 和证明 π 。他使用验证密钥 vk ，通过一个高效的**双线性配对 (Pairing)** 运算来检查证明的有效性。这个运算允许验证者在不直接计算复杂多项式的情况下，验证 $A(x) \cdot B(x) - C(x) = H(x) \cdot Z(x)$ 这个关系是否成立。

双线性配对 e 的一个关键性质是：

$$e(g_1^a, g_2^b) = e(g_1, g_2)^{ab} \quad (3)$$

其中 g_1 和 g_2 是椭圆曲线群的生成元。这个性质使得验证者可以在加密空间中高效地验证多项式乘法关系，这是实现“简洁”验证的核心。

3 实验

为完成本次实验，我们将编写如下四个模块的代码。

3.1 common.hpp: 算术电路与 R1CS 约束定义

```

1 #include <libsark/common/default_types/r1cs_gg_ppzksnark_pp.hpp>
2 #include <libsark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>
3 #include <libsark/gadgetlib1/pb_variable.hpp>
4
5 using namespace libsark;
6 using namespace std;
7
8 // 定义使用的有限域
9 typedef libff::Fr<default_r1cs_gg_ppzksnark_pp> FieldT;
10
11 // 定义创建面包板的函数
12 protoboard<FieldT> build_protoboard(int* secret)
13 {
14     // 初始化曲线参数
15     default_r1cs_gg_ppzksnark_pp::init_public_params();
16     // 创建面包板
17     protoboard<FieldT> pb;
18
19     // 定义所有需要外部输入的变量以及中间变量
20     pb_variable<FieldT> out;    // 公开输出
21     pb_variable<FieldT> x;      // 私密输入 x
22     pb_variable<FieldT> sym_1;  // 中间变量 x^2
23     pb_variable<FieldT> y;      // 中间变量 x^3
24
25     // 将各个变量与 protoboard 连接
26     out.allocate(pb, "out");
27     x.allocate(pb, "x");
28     sym_1.allocate(pb, "sym_1");
29     y.allocate(pb, "y");
30
31     // 声明与 pb 连接的前 1 个变量是 public 的 (即 out)
32     pb.set_input_sizes(1);
33
34     // 添加 R1CS 约束

```

```

35 // 1.  $x \cdot x = \text{sym\_1}$ 
36 pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x, x, sym_1));
37 // 2.  $\text{sym\_1} \cdot x = y$ 
38 pb.add_r1cs_constraint(r1cs_constraint<FieldT>(sym_1, x, y));
39 // 3.  $(y + x + 5) \cdot 1 = \text{out}$ 
40 pb.add_r1cs_constraint(r1cs_constraint<FieldT>(y + x + 5, 1, out));
41
42 // 为变量赋值：无论是生成证明还是验证，公开结果 out 都是 35
43 pb.val(out) = 35;
44
45 // 证明者在生成证明阶段传入私密输入
46 if (secret != NULL)
47 {
48     pb.val(x) = secret[0];
49     pb.val(sym_1) = secret[1];
50     pb.val(y) = secret[2];
51 }
52
53 return pb;
54 }

```

1. 变量分配与电路拓扑

该文件定义了系统的核心逻辑，通过 `protoboard`（面包板）类管理电路变量：

- **变量定义**：使用 `pb_variable` 定义了 `out`, `x`, `sym_1`, `y`。在算术电路模型中，这些变量对应电路的导线（Wires）。
- **输入分类**：`x`, `sym_1`, `y` 被归类为辅助输入（Auxiliary Input），即 **私密见证（Witness）**；而 `out` 通过 `pb.set_input_sizes(1)` 被声明为主输入（Primary Input），即 **公开声明（Public Statement）**。

2. 约束构建（数学原理映射）

R1CS 的标准形式要求约束满足 $(A \cdot s) \times (B \cdot s) = (C \cdot s)$ 。代码通过 `add_r1cs_constraint` 将高阶多项式降阶为以下二次约束：

- **平方约束**：`pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x, x, sym_1));`
对应数学逻辑： $x \cdot x = x^2$ 。
- **立方约束**：`pb.add_r1cs_constraint(r1cs_constraint<FieldT>(sym_1, x, y));`
对应数学逻辑： $x^2 \cdot x = x^3$ 。
- **多项式求和约束**：`pb.add_r1cs_constraint(r1cs_constraint<FieldT>(y + x + 5, 1, out));`
对应数学逻辑： $(x^3 + x + 5) \cdot 1 = \text{out}$ 。这里我们利用与常数 1 相乘，将线性加法运算转化为 R1CS 所需的乘法门形式。

3.2 mysetup.cpp：可信设置

```

1 #include <libsark/common/default_types/r1cs_gg_ppzksnark_pp.hpp>
2 #include <libsark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>

```

```

3 #include <fstream>
4 #include "common.hpp"
5
6 using namespace libsnark;
7 using namespace std;
8
9 int main()
10 {
11     // 构造面包板
12     protoboard<FieldT> pb = build_protoboard(NULL);
13     const r1cs_constraint_system<FieldT> constraint_system =
        pb.get_constraint_system();
14
15     // 生成证明密钥和验证密钥
16     const r1cs_gg_ppzksnark_keypair<default_r1cs_gg_ppzksnark_pp> keypair =
        r1cs_gg_ppzksnark_generator<default_r1cs_gg_ppzksnark_pp>(constraint_system);
17
18     // 保存证明密钥到文件 pk.raw
19     fstream pk("pk.raw", ios_base::out);
20     pk << keypair.pk;
21     pk.close();
22
23     // 保存验证密钥到文件 vk.raw
24     fstream vk("vk.raw", ios_base::out);
25     vk << keypair.vk;
26     vk.close();
27
28     return 0;
29 }

```

该模块负责执行 zk-SNARKs 的**预处理阶段**。通过 `r1cs_gg_ppzksnark_generator` 生成全局公共参数：

- **证明密钥 (Proving Key, pk)**：包含用于将 Witness 编码为加密证明的椭圆曲线基点。
- **验证密钥 (Verification Key, vk)**：包含用于双线性配对检查的公共参数，体积远小于 pk 。

注：此步骤产生的“毒性废物”（随机数）必须被销毁，否则协议安全性将崩溃。

3.3 myprove.cpp：证明生成

```

1 #include <libsnark/common/default_types/r1cs_gg_ppzksnark_pp.hpp>
2 #include <libsnark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>
3 #include <fstream>
4 #include "common.hpp"
5
6 using namespace libsnark;
7 using namespace std;
8

```

```

9  int main()
10 {
11     // 输入秘密值 x (本实验中正确解应该是 3)
12     int x;
13     cout << "请输入秘密解 x: ";
14     cin >> x;
15
16     // 为私密输入提供具体数值
17     int secret[3];
18     secret[0] = x;           // x
19     secret[1] = x * x;       // x^2
20     secret[2] = x * x * x;   // x^3
21
22     // 构造面包板
23     protoboard<FieldT> pb = build_protoboard(secret);
24
25     // 加载证明密钥
26     fstream f_pk("pk.raw", ios_base::in);
27     rlcs_gg_ppzksnark_proving_key<libff::default_ec_pp> pk;
28     f_pk >> pk;
29     f_pk.close();
30
31     // 生成证明
32     const rlcs_gg_ppzksnark_proof<default_rlcs_gg_ppzksnark_pp> proof =
        rlcs_gg_ppzksnark_prover<default_rlcs_gg_ppzksnark_pp>(pk,
            pb.primary_input(), pb.auxiliary_input());
33
34     // 将生成的证明保存到 proof.raw 文件
35     fstream pr("proof.raw", ios_base::out);
36     pr << proof;
37     pr.close();
38
39     cout << "已生成证明至 proof.raw" << endl;
40     return 0;
41 }

```

证明者在获取秘密解 $x = 3$ 后，计算完整的见证向量 $s = (1, out, x, x^2, x^3)$ 。

- **零知识处理**：证明算法利用加密技术将 s 映射到椭圆曲线群元素上。生成的 `proof.raw` 仅包含曲线上的点，有效地隐藏了原始输入 x 的信息。

3.4 myverify.cpp: 证明验证

```

1  #include <libsark/common/default_types/rlcs_gg_ppzksnark_pp.hpp>
2  #include <libsark/zk_proof_systems/ppzksnark/rlcs_gg_ppzksnark/rlcs_gg_ppzksnark.hpp>
3  #include <fstream>
4  #include "common.hpp"
5
6  using namespace libsark;

```



```

7 using namespace std;
8
9 int main()
10 {
11     // 构造面包板 (out 已经默认赋值为 35)
12     protoboard<FieldT> pb = build_protoboard(NULL);
13
14     // 加载验证密钥
15     fstream f_vk("vk.raw", ios_base::in);
16     r1cs_gg_ppzksnark_verification_key<libff::default_ec_pp> vk;
17     f_vk >> vk;
18     f_vk.close();
19
20     // 加载证明方生成的证明
21     fstream f_proof("proof.raw", ios_base::in);
22     r1cs_gg_ppzksnark_proof<libff::default_ec_pp> proof;
23     f_proof >> proof;
24     f_proof.close();
25
26     // 进行验证
27     bool verified =
28         r1cs_gg_ppzksnark_verifier_strong_IC<default_r1cs_gg_ppzksnark_pp>(vk,
29             pb.primary_input(), proof);
30
31     cout << "验证结果: " << verified << endl;
32     return 0;
33 }

```

- **核心逻辑**: 验证者无需重复 $x^3 + x + 5$ 的计算, 而是通过 **双线性配对**检查证明中各个元素之间的数学关系是否符合 vk 预定义的约束。
- **验证算式简述**: 检查 $e(\pi_A, \pi_B) = e(\alpha_1, \beta_2) \cdot e(\sum_{id} \frac{L_i(input)}{id}, \gamma_2) \cdot e(\pi_C, \delta_2)$ 。

4 结果及验证

编写完上一节四个模块的代码后, 我们需要对其编译并运行。我们需要将以上四个文件加入到 /Libsnark/libsnark_abc-master/src 目录下的 CMakeLists.txt 中, 具体操作参见教材。随后我们执行如下指令:

```

1 cmake ..
2 make
3 cd src
4 ./mysetup
5 ./myprove
6 3
7 ./myverify

```

得到如下结果:

```
kxh@ubuntu: ~/Desktop/Libsnark/libsnark_abc-master/build/src
G2 elements in proof: 1
Proof size in bits: 1019
已生成证明至 proof.raw
kxh@ubuntu:~/Desktop/Libsnark/libsnark_abc-master/build/src$ ./myverify
(enter) Call to rics_gg_ppzksnark_verifier_strong_IC [ ] (1776937650.1305s x0.00 from start)
(enter) Call to rics_gg_ppzksnark_verifier_process_vk [ ] (1776937650.1307s x0.00 from start)
(enter) Call to alt_bn128_ate_precompute_G2 [ ] (1776937650.1307s x0.00 from start)
(leave) Call to alt_bn128_ate_precompute_G2 [0.0018s x1.00] (1776937650.1325s x0.00 from start)
(enter) Call to alt_bn128_ate_precompute_G2 [ ] (1776937650.1326s x0.00 from start)
(leave) Call to alt_bn128_ate_precompute_G2 [0.0028s x1.00] (1776937650.1346s x0.00 from start)
(leave) Call to rics_gg_ppzksnark_verifier_process_vk [0.0041s x1.00] (1776937650.1347s x0.00 from start)
(enter) Call to rics_gg_ppzksnark_online_verifier_strong_IC [ ] (1776937650.1348s x0.00 from start)
(enter) Call to rics_gg_ppzksnark_online_verifier_weak_IC [ ] (1776937650.1348s x0.00 from start)
(enter) Accumulate input [ ] (1776937650.1349s x0.00 from start)
(leave) Accumulate input [0.0001s x0.99] (1776937650.1350s x0.00 from start)
(enter) Check if the proof is well-formed [ ] (1776937650.1351s x0.00 from start)
(leave) Check if the proof is well-formed [0.0001s x0.99] (1776937650.1352s x0.00 from start)
(enter) Online pairing computations [ ] (1776937650.1353s x0.00 from start)
(enter) Check QAP divisibility [ ] (1776937650.1353s x0.00 from start)
(enter) Call to alt_bn128_ate_precompute_G1 [ ] (1776937650.1353s x0.00 from start)
(leave) Call to alt_bn128_ate_precompute_G1 [0.0000s x1.02] (1776937650.1354s x0.00 from start)
(enter) Call to alt_bn128_ate_precompute_G2 [ ] (1776937650.1354s x0.00 from start)
(leave) Call to alt_bn128_ate_precompute_G2 [0.0025s x1.00] (1776937650.1379s x0.00 from start)
(enter) Call to alt_bn128_ate_precompute_G1 [ ] (1776937650.1380s x0.00 from start)
(leave) Call to alt_bn128_ate_precompute_G1 [0.0000s x1.00] (1776937650.1380s x0.00 from start)
(enter) Call to alt_bn128_ate_precompute_G1 [ ] (1776937650.1381s x0.00 from start)
(leave) Call to alt_bn128_ate_precompute_G1 [0.0000s x1.01] (1776937650.1381s x0.00 from start)
(enter) Call to alt_bn128_ate_miller_loop [ ] (1776937650.1382s x0.00 from start)
(leave) Call to alt_bn128_ate_miller_loop [0.0051s x1.00] (1776937650.1433s x0.00 from start)
(enter) Call to alt_bn128_ate_double_miller_loop [ ] (1776937650.1434s x0.00 from start)
(leave) Call to alt_bn128_ate_double_miller_loop [0.0081s x1.00] (1776937650.1516s x0.00 from start)
(enter) Call to alt_bn128_final_exponentiation [ ] (1776937650.1517s x0.00 from start)
(enter) Call to alt_bn128_final_exponentiation_first_chunk [ ] (1776937650.1518s x0.00 from start)
(leave) Call to alt_bn128_final_exponentiation_first_chunk [0.0003s x1.01] (1776937650.1521s x0.00 from start)
(enter) Call to alt_bn128_final_exponentiation_last_chunk [ ] (1776937650.1522s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1776937650.1523s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0027s x1.00] (1776937650.1549s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1776937650.1552s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0030s x1.00] (1776937650.1581s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1776937650.1583s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0032s x1.00] (1776937650.1615s x0.00 from start)
(leave) Call to alt_bn128_final_exponentiation_last_chunk [0.0120s x1.00] (1776937650.1642s x0.00 from start)
(leave) Call to alt_bn128_final_exponentiation [0.0126s x1.00] (1776937650.1643s x0.00 from start)
(leave) Check QAP divisibility [0.0291s x1.00] (1776937650.1644s x0.00 from start)
(leave) Online pairing computations [0.0292s x1.00] (1776937650.1645s x0.00 from start)
(leave) Call to rics_gg_ppzksnark_online_verifier_weak_IC [0.0297s x1.00] (1776937650.1645s x0.00 from start)
(leave) Call to rics_gg_ppzksnark_online_verifier_strong_IC [0.0298s x1.00] (1776937650.1646s x0.00 from start)
(leave) Call to rics_gg_ppzksnark_verifier_strong_IC [0.0342s x1.00] (1776937650.1646s x0.00 from start)
验证结果: 1
kxh@ubuntu:~/Desktop/Libsnark/libsnark_abc-master/build/src$
```

图 4.1: 验证结果

5 Github 链接

由于本次实验配置流程较为复杂, Github 链接仅提供编写的四个文件。
编写的四个文件访问[Github 链接](#)获取。